

CSA-0606- DESIGN AND ANALYSIS OF ALGORITHMS

CO 2 – AT 2

Q33. Library System – Linear Search Analysis

1. Analyze the Given Scenario

A library stores thousands of books, each identified by a unique **Book ID**. Because new books are frequently added and old books are removed or updated, the Book IDs are stored in an **unsorted list**. Users search for books by entering the Book ID.

Since the list is **unsorted**, the system cannot directly use Binary Search. Instead, it performs a **Linear Search**, which checks each Book ID one by one until the required Book ID is found or the list ends.

(a) How Linear Search is Applied

Suppose the Book IDs are:

[205, 310, 108, 425, 512, 180, 620]

User searches for:

Book ID = 512

The system compares:

- Compare 205 → Not Found
- Compare 310 → Not Found
- Compare 108 → Not Found
- Compare 425 → Not Found
- Compare 512 → Found

Book details are displayed immediately after the match is found.

If the Book ID does not exist, the system checks all records and displays "**Book Not Found.**"

2. Identify User Requirements

The library users expect the following features:

Functional Requirements

- Search books using Book ID.
- Display complete book information.
- Add new books.
- Delete old books.
- Update book details.
- Show availability status.
- Display "Book Not Found" for invalid IDs.

Non-Functional Requirements

- Fast response time.
- Easy-to-use interface.
- Accurate search results.
- Secure storage of records.
- Reliable performance.

3. Suggest Suitable Technologies

Frontend

- HTML
- CSS
- JavaScript
- Bootstrap (optional)

Backend

- Python (Flask/Django)
- PHP

- Java

Database

- MySQL
- SQLite
- PostgreSQL

Development Tools

- VS Code
- XAMPP
- Python IDLE

4. Design the Webpage

Search Page

LIBRARY MANAGEMENT SYSTEM

Book ID:

[]

[Search]

Book Details

Book ID:

Book Name:

Author:

Publisher:

Status:

Database Design

Table: Books

Field	Data Type	Description
Book_ID	INT (Primary Key)	Unique Book ID
Book_Name	VARCHAR	Book Name
Author	VARCHAR	Author Name
Publisher	VARCHAR	Publisher
Category	VARCHAR	Book Category
Status	VARCHAR	Available/Issued

Sample Records

Book ID	Book Name	Author	Status
205	Python Basics	John	Available
310	Data Structures	David	Issued
108	DBMS	Smith	Available
425	Operating Systems	James	Available
512	Machine Learning	Alice	Issued

5. Explain Implementation Steps

Step 1

Store all book details in the database.

↓

Step 2

User enters Book ID.

↓

Step 3

Application retrieves all Book IDs.

↓

Step 4

Apply Linear Search.

↓

Step 5

Compare each Book ID with user input.

↓

Step 6

If Match

Display Book Details

Else

Continue Searching

↓

Step 7

If end of list reached

Display

"Book Not Found"

6. Justify Design Decisions

Why Linear Search?

- Book IDs are not sorted.
- Very easy to implement.
- No preprocessing required.
- Supports frequent insertions and deletions.
- Suitable for small and medium-sized libraries.

Why Unsorted List?

Because

- Books are added every day.
- Books are removed frequently.
- Sorting after every update wastes time.
- Updating records becomes easier.

7. Performance Evaluation

(b) Complexity Analysis

Let

n = Number of Books

Best Case

Book is the first record.

Comparisons = 1

Time Complexity

$O(1)$

Example

Search = 205

205 found immediately.

Average Case

Book is somewhere in the middle.

Comparisons

$n/2$

Time Complexity

$O(n)$

Example

Search = 425

Comparisons

205

310

108

425

Worst Case

Book is the last record or absent.

Comparisons

n

Time Complexity

$O(n)$

Example

Search = 620

Need to check every Book ID.

Comparison Table

Case	Comparisons	Time Complexity
------	-------------	-----------------

Best	1	$O(1)$
------	---	--------

Average	$n/2$	$O(n)$
---------	-------	--------

Worst	n	$O(n)$
-------	-----	--------

8. Would Sorting Improve System Efficiency?

Yes.

If the books are sorted,

Example

108

180

205

310

425

512

620

The system can use **Binary Search**.

Binary Search repeatedly divides the list into two halves, reducing the number of comparisons.

Binary Search Complexity

Case	Complexity
------	------------

Best	$O(1)$
------	--------

Average	$O(\log n)$
---------	-------------

Worst	$O(\log n)$
-------	-------------

Advantages of Sorting

- Faster searching.

- Fewer comparisons.
- Efficient for large databases.
- Suitable when searches are more frequent than updates.

Disadvantages of Sorting

- Every insertion may require repositioning records.
- Every deletion may require maintaining sorted order.
- Sorting consumes additional processing time.
- Not efficient when updates occur frequently.

Comparison Between Linear Search and Binary Search

Feature	Linear Search	Binary Search
Requires Sorted Data	No	Yes
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Easy to Implement	Yes	Yes
Suitable for Frequent Updates	Yes	No
Suitable for Large Data	No	Yes

9. Overall Performance Evaluation

For a small library with frequent additions and deletions:

- Linear Search performs adequately.
- No need to maintain sorted order.

For a large library where searches occur much more often than updates:

- Sorting the records and using Binary Search provides significantly faster search performance.
- Although sorting introduces maintenance overhead, the reduction in search time makes it worthwhile for search-intensive systems.

10. Conclusion

- The library system stores Book IDs in an **unsorted list** because books are frequently added, deleted, or updated.
- **Linear Search** is the most suitable searching technique in this scenario because it works directly on unsorted data and is easy to implement.
- The best-case complexity is **$O(1)$** , while the average and worst-case complexities are **$O(n)$** .
- Sorting the Book IDs enables the use of **Binary Search**, reducing the search complexity to **$O(\log n)$** .
- However, maintaining a sorted list increases the cost of insertions and deletions.
- Therefore, Linear Search is the preferred choice for frequently changing library records, whereas Binary Search is more suitable for large, mostly static collections where searching is performed much more often than updates.